

Lecture - 30

Object oriented programming (OOPS)

OOPS : { Class, Object }

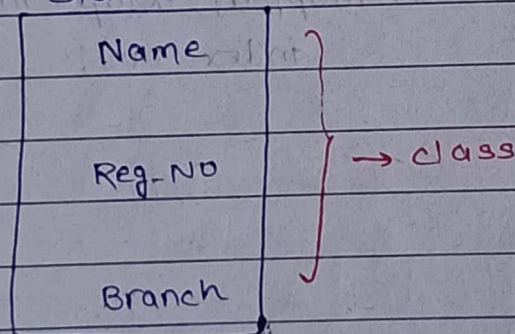
We will talk about class and object in this lecture.

Class : It is a userdefined datatype. It is a blueprint for the object. It is a logical construct.

Object : It is an instance of a class.

Object can occupy space in memory.

Student → Class Name



Earlier, if I have 10 student and we want to store the information of student like name, Reg-No and Branch, then we have made an array for all which we want to store.

```
String name[10];
```

```
int Reg-No[10];
```

```
String Branch[10];
```

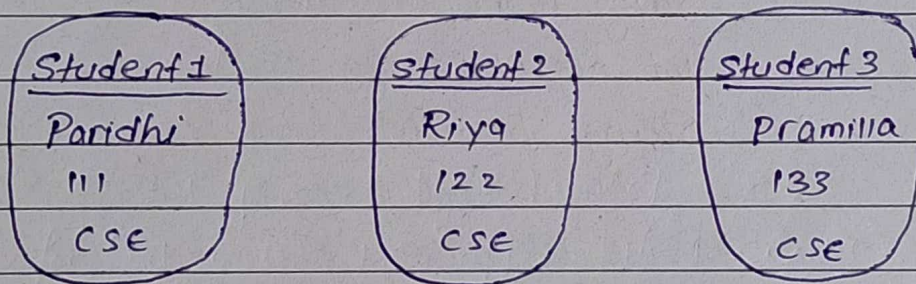

We have to make 3 array for storing.

Accessing the data of 1 student can be difficult that we have to access all the array.

If we want to add one more data then we have to add a different array.

→ Can we make it easier to store the data?

Yes, if data of a student can be stored in a object. then it is easier to access.



By this way we can access easily.

→ class name (user defined datatype)

→

```
Class Student {
    String name;
    int Reg-No;
    String Branch;
}
```

How to access data of 1 student:

```
int main() {
    Student Std1;
    Std1.name = "Paridhi";
    Std1.Reg_No = 111;
    Std1.Branch = "CSE";
}
```



```

    cout << st1.Reg-NO;
    return 0;
}

```

By default class is private.

So, in above program it gives error.

We have to make the class as public.

```

class student {
    public:
        String name;
        int Reg-NO;
        String branch;
}

```

Now, the code works.

Access specifier :

- public
- private (By default)
- protected

class

- Behaviour
- state / property

property like :

Memory

Color

Company

Behaviour / Action :

→ click : photo clicked

→ setting : Execute.

We can also make function inside class

↳ to show behaviour.

→ let us take an example of Bank:

```
class Bank {  
    public :  
        int balance;  
        int name;  
        void balance_check () {  
            cout << balance << " ";  
        };  
        void withdraw () {  
            balance -= 100;  
            cout << " ₹ 100 Withdrawn ";  
            cout << " New Balance " << balance;  
        };  
}  
  
int main () {  
    Bank Customer1;  
    Customer1.balance = 500;  
    Customer1.name = "Riya";  
    Customer1.balance_check();  
    Customer1.withdraw();  
    return 0;  
}
```


In, this program there is a problem:
 user can self modify the balance, if
 user have access then it is not safe.
 We can change the data by self.
 So, what we do?

make balance as private:
 And make public to those which is required:-

```
class Bank {
    int balance;
    String name;
}
```

} → private

public:

```
void setValue (int amount, String names) {
    balance = amount;
    name = names;
}
```

```
void checkBalance () {
    cout << balance;
}
```

```
void Withdraw () {
    balance -= 100;
    cout << "₹ 100 Withdrawn";
    cout << balance;
}
```

```
}
int main () {
    Bank customer1;
    customer1.setValue (10000, "Paridhi");
    customer1.checkBalance ();
    customer1.Withdrawn ();
}
```


This can also create problem. Because user or anyone can update the balance easily.

So, we have to make this like, whenever we want to update balance, we cannot update it directly. We can update it with the permission of bank employee and they can verify it.

```
class Bank {
    int Balance;
    String name;
```

In this the name & password is of clerk. If clerk can login and update then the balance is updated.

```
public :
```

```
    void setValue (int amount, String name,
                  String password) {
        if (Name == "clerk-Name" &&
            password == "password-clerk") {
            Balance += amount;
        }
```

```
    void checkBalance () {
        cout << Balance;
    }
```

```
    void Withdraw () {
        Balance -= ₹100;
        cout << "₹ 100 Withdrawn";
        cout << Balance;
    }
```

```
}
```

```
int main () {
```

```
    Bank Customer;
```


function called
by clerk

Name is of
clerk

clerk
password
Date

Customer1.setValue (1000, "Rajesh", "*****");

Customer1.checkBalance();

Customer1.withdrawn();

return 0;

}

* ANOTHER EXAMPLE :

```
class car {
```

```
    int Mileage;
```

```
    int cost;
```

```
    public:
```

```
    void IncMileage() {
```

```
        Mileage += 2;
```

```
    }
```

```
int main() {
```

```
    car obj;
```

```
    cout << sizeof(obj);
```

// return 8.

```
    return 0;
```

```
}
```

```
class car {
```

```
    int Mileage;
```

```
    char cost;
```

// return 8

```
}
```

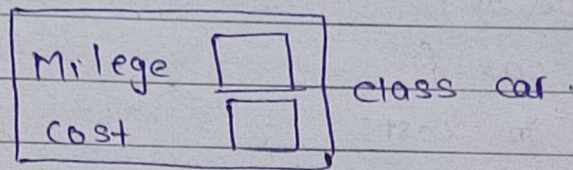
Why both gives 8?

int + int = 8 (sm ayya)

int+char = 8 (How)

It happens by the padding & alignment.

Teacher's Signature.....



Default Value :

→ If we don't give any value then it take default value which is done by Constructor.

जब तक user input नहीं देता है तब तक उसे initial value आ जाए तो वो Constructor करता है।

```

class car {
    int Cost;
    int Mileage;

```

The name of
Constructor is same
as class.

```

public :
    car() {

```

Cost = 2,00,000;

Mileage = 30;

```

    }

```

```

int main() {

```

```

    car obj;

```

```

}

```

If you don't made any Constructor then Computer made a default Constructor and put random value to it.

Also possible by:

```
class car {
    int Cost;
    int Milege;
    public:
```

Constructor → `car (int price, int speed) {`
`Cost = price;`
`Milege = speed;`
`cout << "Hello Constructor";`
`};`

```
void print() {
    cout << Cost << " " << Milege;
}
```

```
}
```

```
int main() {
    car Adi (20, 40);
    Adi.print();
}
```

If we write Constructor as:

```
car (int Cost, int Milege) {
```

This gives error

because it get Confused.

```
Cost = Cost;
Milege = Milege;
cout << "Hello Constructor";
```

int can be correct by:

`this → Cost = Cost;`

`this → Milege = Milege;`


```
int main () {
```

```
    Student *p = new Student();
```

```
    *p.reg = 10;
```

```
    p->reg = 10;
```

} Both are same.

→ class car {

```
    public:
```

```
        int cost;
```

```
        int Milege;
```

```
};
```

```
int main () {
```

```
    car *p = new car;
```

```
    (*p).cost = 20;
```

```
    p->milege = 25;
```

```
    cout << p.cost;
```

* Destructor:

जो ग्री memory occupy की हुई है उसको Release कर दो। use होने के बाद।

```
class car {
```

```
    public:
```

```
        int cost;
```

```
        int Milege;
```

```
    car() {
```

```
        cout << "Constructor";
```

```
    }
```

```
    ~car() {
```

```
        cout << "Destructor";
```

```
    }
```

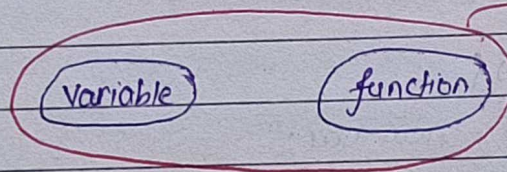
```
}
```



```
int main() {  
    car *p = new car;  
    delete p;  
    return 0;  
}
```

* Encapsulation :

Combination of variable & function



→ Encapsulation.

* Abstraction :

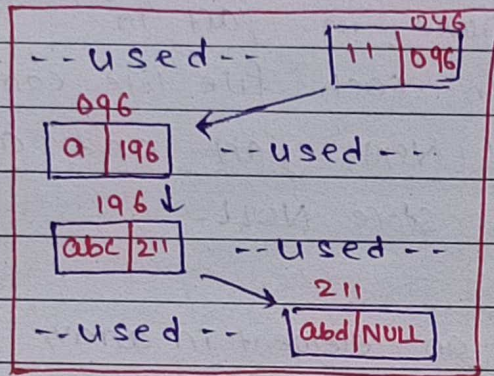
It can hide the data which are not used.
Show only that data which you need.

By this we can achieve Code Reusability.

Linked List

Memory

Address = 046



user demand for 4 space for storing.

In the memory, It can't available at one place means Continuous.

So, what we do. We give non continuous Memory.

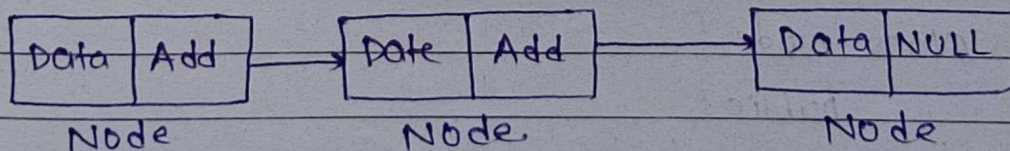
So, how he know where the next memory occur.

Either we give the address of all Memory space.

It is very tedious task to remember.

Then, what we do. We can discover linked list.

In linked ^{list} we can give address of 1st space and the second space address can be stored in first space.



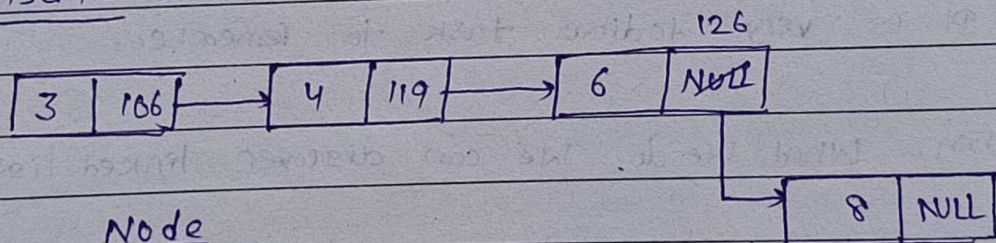
You have a movie of 4 GB.

In your laptop there is space of 4GB vacant, but Not at one place.

So, we store the movie in part in different location and in each file we can write the address of next part and at the last part we store NULL.

- If we want to insert an element in array, then it is very difficult.
- But in linked list we simply add an element and insert the index.

Insertion :



Node

int data
Node *next

↓
We want to add

Code :

```

class Node {
public:
    int data;
    Node * next;
}
  
```